



TECNOLÓGICO NACIONAL DE MÉXICO

Instituto Tecnológico de Morelia

Departamento de Sistemas y Computación

Programacion Web

Semestre: 6 Grupo: C

PROYECTO FINAL PARTE 3: IMPLEMENTACION DE JAVASCRIPT Y FRAMEWORK TAILWIND CSS

Profesor:

Juan Carlos Olivares Rojas

Alumno:

Mendoza Bryan

Morelia, Michoacán

24 de abril de 2026



Índice

1. Introducción	3
2. Arquitectura del sistema	3
3. Desarrollo del Frontend	3
3.1. Topología interactiva	3
3.2. Inventario de dispositivos	4
3.3. Soporte técnico	4
3.4. Contacto	5
4. Desarrollo del Backend	5
4.1. Configuración del servidor	5
4.2. Conexión a la base de datos	5
5. Base de datos	5
5.1. Tablas principales	5
5.2. Ejemplo de creación de tabla	6
6. API REST	6
6.1. Endpoints de dispositivos	6
6.2. Endpoints de tickets	6
6.3. Ejemplo de creación de ticket	6
7. Pruebas con Postman	6
8. Opiniones sobre este proyecto	7
9. Conclusión general	7



1. Introducción

En el presente proyecto se desarrolló un sistema web de monitoreo de red, cuyo objetivo es representar de manera visual e interactiva el estado de los dispositivos dentro de una infraestructura de red local (LAN).

El sistema permite visualizar la topología de red, gestionar dispositivos, reportar incidencias técnicas y almacenar información en una base de datos. Para su desarrollo se utilizaron tecnologías modernas como HTML, CSS, JavaScript en el frontend, y Node.js con Express y PostgreSQL en el backend.

Este proyecto fue realizado como parte de la materia de Programación Web, integrando conocimientos de desarrollo web completo (Full Stack).

2. Arquitectura del sistema

El sistema está dividido en dos partes principales:

- **Frontend:** Interfaz visual desarrollada con HTML, CSS y JavaScript.
- **Backend:** API REST desarrollada con Node.js y Express.
- **Base de datos:** PostgreSQL gestionado mediante Docker.

El flujo general es el siguiente:

1. El usuario interactúa con la interfaz web.
2. El frontend envía solicitudes HTTP (fetch) al backend.
3. El backend procesa la solicitud.
4. Se consulta o modifica la base de datos PostgreSQL.
5. Se devuelve una respuesta en formato JSON.

3. Desarrollo del Frontend

El frontend del sistema está compuesto por varias vistas principales:

3.1. Topología interactiva

Se desarrolló una representación gráfica de la red donde se visualizan dispositivos como:

- Router
- Switch
- Servidor
- PCs



- Access Point

Cada dispositivo muestra su estado:

- Online
- Offline
- Degradación

Se implementaron mejoras visuales como:

- Ajuste de tamaños de texto
- Colores dinámicos según estado
- Distribución lógica tipo árbol

3.2. Inventario de dispositivos

Se creó una tabla dinámica que muestra:

- Nombre del dispositivo
- Dirección IP
- Dirección MAC
- Tipo de dispositivo
- Ubicación
- Estado

Además, se implementaron:

- Filtros de búsqueda
- Cambio de estado mediante menú desplegable
- Botones de actualización de vista

3.3. Soporte técnico

Se desarrolló un formulario para generar tickets de incidencias con los siguientes campos:

- ID del dispositivo
- Correo electrónico
- Tipo de incidencia
- Prioridad
- Descripción



También se muestra una lista de tickets generados, con opción de cerrarlos.

3.4. Contacto

Se implementó una vista de información del desarrollador, incluyendo:

- Datos personales
- Perfiles profesionales
- Formulario de contacto
- Historial de mensajes enviados

4. Desarrollo del Backend

El backend fue desarrollado utilizando Node.js con el framework Express.

4.1. Configuración del servidor

El archivo principal define rutas básicas y conexión a la base de datos:

```
1 app.get("/api/test-db", async (req, res) => {  
2   const result = await pool.query("SELECT NOW()");  
3 });
```

Este endpoint permite verificar la conexión con PostgreSQL.

4.2. Conexión a la base de datos

Se utilizó el módulo pg para conectarse a PostgreSQL mediante variables de entorno:

```
1 const pool = new Pool({  
2   host: process.env.DB_HOST,  
3   user: process.env.DB_USER,  
4   password: process.env.DB_PASSWORD,  
5 });
```

5. Base de datos

Se utilizó PostgreSQL ejecutado en un contenedor Docker.

5.1. Tablas principales

- dispositivos
- tickets
- mensajes_contacto



- usuarios

5.2. Ejemplo de creación de tabla

```
1 CREATE TABLE dispositivos (  
2   id SERIAL PRIMARY KEY,  
3   nombre VARCHAR(100),  
4   ip VARCHAR(50),  
5   mac VARCHAR(50),  
6   tipo VARCHAR(50),  
7   ubicacion VARCHAR(100),  
8   estado VARCHAR(20)  
9 );
```

6. API REST

Se implementaron endpoints para manejar dispositivos y tickets.

6.1. Endpoints de dispositivos

- GET /api/devices
- GET /api/devices/:id

6.2. Endpoints de tickets

- GET /api/tickets
- GET /api/tickets/:id
- POST /api/tickets
- PATCH /api/tickets/:id/close

6.3. Ejemplo de creación de ticket

```
1 {  
2   "dispositivo": "Router-Core-01",  
3   "correo": "user@example.com",  
4   "tipo": "hardware",  
5   "prioridad": "Alta",  
6   "descripcion": "Falla de conexion"  
7 }
```

7. Pruebas con Postman

Se realizaron pruebas utilizando Postman para verificar:

- Conexión a la base de datos



- Inserción de registros
- Consulta de datos
- Manejo de errores

Se detectaron errores como:

- Problemas de autenticación
- Tablas no existentes
- Puertos en uso

Los cuales fueron resueltos mediante:

- Reconfiguración de Docker
- Reinicio de contenedores
- Ejecución de scripts SQL

8. Opiniones sobre este proyecto

Este proyecto me permitió comprender de manera práctica cómo funciona una arquitectura web completa, desde la interfaz hasta la base de datos. Aprendí a trabajar con APIs REST, manejar errores reales de conexión y estructurar un backend profesional.

También reforcé el uso de Docker y PostgreSQL, lo cual considero muy importante para proyectos reales. Me siento satisfecho con el resultado y considero que es una base sólida para continuar con funcionalidades más avanzadas como autenticación de usuarios.

9. Conclusión general

El desarrollo de este sistema demuestra la integración de múltiples tecnologías en un entorno realista. Se logró construir una aplicación funcional que simula un sistema de monitoreo de red, con visualización, gestión de datos y comunicación entre frontend y backend.

Este tipo de proyectos fortalece habilidades esenciales en el desarrollo web moderno, preparando al estudiante para enfrentar escenarios reales en la industria.